

Desarrollo de Software en Sistemas Distribuidos

Cátedra: Azcurra – Amaro - Tomczak.

Cuatrimestre: Segundo

Año: 2025

TRABAJO PRÁCTICO

RPC - Empuje solidario

Grupo 5: Integrantes

Nombre	Martin Bombara	Daniel Geier	Sebastián Marioni	Luciano Moliterno
Tareas Realizadas	- Creado cliente gRPC python - Creación y propagación JWT - Cifrado del password y envío de email - Servicio de eventos - Implementación de Docker	- Backend de Usuarios (usando Java Spring Boot) - Uso de Protobuf para la creación de usuario.proto - Backend de Inventario de Donaciones (usando Java Spring Boot) (en desarrollo en una rama "servicio-de-donaciones")	Frontend de: Usuarios: Actualizar, Crear, Dar de baja (Lógica) Login de usuarios Frontend de Eventos: Actualizar, Crear, Eliminar, Asignar/ Eliminar voluntarios (Está el desarrollo subido en una rama llamada abm-eventos pero no está conectado con el backend aun. En el final del documento hay otro docs con las capturas correspondientes)	- view login - view registro usuarios - roles usuarios - interfaz proto usuarios - schema db users - db users - login de un usuario agregue en una rama aparte "inventario-grpc" lo siguiente: nuevo microservicio inventario + gateway flask (expone inventario) + docker compose actualizado para levantar inventario y conectar el gateway por gRPC
DNI	40812039	40229425	42281606	40238958

Repositorio Github

<https://github.com/DanyGeier/Grupo-TN-G>

Pendientes

- *Validación del JWT en todos los servicios*
- *Integrar servicio de donaciones*
- *Formato del email*

Estrategia de resolución

Tecnologías utilizadas

- Frontend WEB: **TS con React + Vite**
- Cliente Rest <-> gRPC : **Python**
- Servicios backend (Usuarios, Eventos, Donaciones) : **Java con Spring Boot**
- DB Servicio Usuarios y Donaciones: **MySQL**
- DB Servicio Eventos: **MongoDB**
- SMTP Envío de emails: **Mailpit**
- Virtualización / Contenedores: **Docker**
- Otras herramientas de desarrollo:
 - **Postman**
 - **Trello**

Descripción de la resolución:

Arquitectura y división de servicios

Pensando en la mantenibilidad y escalabilidad del futuro sistema para empuje solidario, se pensó una arquitectura de microservicios comunicados a través del protocolo gRPC en lugar de Rest como se hace habitualmente. Esta estructura permite independencia entre los microservicios y un manejo más atómico de cada proyecto (No monolitos).

Cada microservicio tiene acceso a una base de datos la cual no puede ser accesada mediante cualquier otro servicio que no sea el servicio principal por el que fue creada (Responsabilidad única)

El frontend web solo se encarga de mostrar las vistas a los usuarios según tengan permitido por su rol y persistir un JWT token generado por el servicio de Usuarios al momento de realizar el login.

Este JWT nos permite manejar la seguridad en nuestra aplicación. Tiene una fecha de expiración y está creado para un usuario específico teniendo datos del mismo y los roles que tiene. En un primer momento es evaluado por el cliente para determinar si no está expirado pero luego tiene que ser validado nuevamente por el servidor para entender los roles del usuario.

Nuestro cliente gRPC en Python actúa como un API Gateway entre Rest y gRPC o entre nuestro frontend y nuestros servicios en backend. También podría decirse que es como un BFF (Backend for frontend) ya que tiene función de agregación de datos para la vista además.

Este cliente define todos los mensajes .proto y servicios disponibles por gRPC que luego serán implementados por los servicios.

Cada servicio es independiente y se encuentra hecho con Java + Spring boot para una simplicidad en el desarrollo gracias a las annotations que permiten conectarse con el cliente y otros servidores gRPC con relativa facilidad.

Para las bases de datos, elegimos mongodb en el servicio de Eventos dada su escalabilidad horizontal y que al ser no relacional y basada en documentos más flexibles nos pueden permitir agregar o modificar campos sin migraciones costosas pensando en un escenario de constante cambio y expansión

Para los otros dos servicios, usamos Mysql dada la facilidad y experiencia en el equipo con bases de datos relacionales y esta en particular además de que sigue y respeta los principios ACID (Atomicity, Consistency, Isolation, and Durability) lo cual nos parece importante en esas bases en particular.

La contraseña es cifrada al momento de crearse un usuario y es enviada por mail por requisito de la consigna. La misma queda almacenada de forma segura en la base de datos.

Cada endpoint valida el rol del usuario antes de permitir la acción correspondiente (por ejemplo, solo el PRESIDENTE puede gestionar usuarios, solo VOCAL puede acceder al inventario, etc.). Esto garantiza que los usuarios solo puedan ejecutar las operaciones para las que tienen permiso, evitando accesos no autorizados y manteniendo la integridad del sistema.

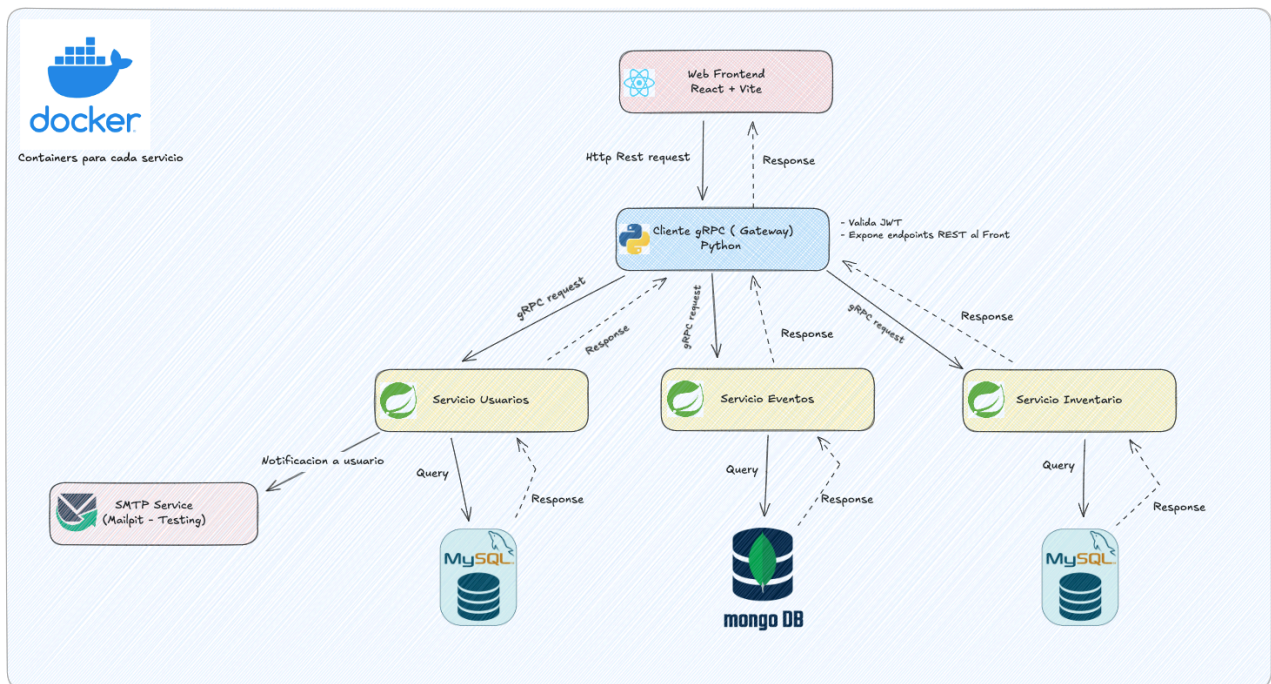
Toda la solución está montada sobre Docker y utilizando Docker compose para manejar los distintos contenedores logramos tener disponible el sistema rápida y fácilmente reduciendo los costos y tiempos de setup en cada una de las configuraciones posibles.

Organización de trabajo/división de actividades

Utilizamos Trello para gestionar tareas y coordinar al equipo. Cada proyecto se organiza en tableros con listas que reflejan el estado de las actividades. Las tarjetas representan tareas individuales, y contienen a los miembros responsables de realizarlas.

<https://trello.com/b/yznApxNZ/temp-tp-distribuidos>

Diagramas



Problemas/Errores

- Utilización de gRPC, ya que la mayoría de los integrantes no tenía experiencia previa con su uso.
- Dificultades con React, al no haberlo usado antes, especialmente en mantener el código limpio y aplicar buenas prácticas de desarrollo.

- Con Protobuf, las generated-sources a veces no se generan en la carpeta target, lo que impide utilizarlas en el backend (al parecer la versión de protobuf que usamos <0.6.1> tiene inconsistencias a la hora de crearla).

Pruebas

Opción 1: Descripción de las pruebas realizadas con las correspondientes capturas de pantalla.

Opción 2: En caso que prefieran hacerlo en video: enlace al video (deberá ser un video con voz, especificando qué pruebas se están ejecutando)

 Capturas de pantalla del front de eventos

Video de pruebas:  Grabación de pantalla 2025-09-21 a la(s) 8.05.54 p. m..mov